

KAI CHATBOT

PROJECT DOCUMENTATION

Production-grade AI Chatbot Platform

FastAPI LangGraph Gemini Pro PostgreSQL ChromaDB Next.js

FastAPI

LangGraph

Gemini Pro

Next.js

PROJECT DOCUMENTATION

1. Project Overview

This project is a scalable, production-grade AI chatbot platform featuring advanced orchestrations including a RAG (Retrieval-Augmented Generation) pipeline, LangGraph-powered Multi-Agent architecture, and Human-in-the-Loop (HITL) approval mechanics. It consists of a FastAPI backend connecting to PostgreSQL (for relational data) and ChromaDB (for vector embeddings), alongside a modern Next.js React frontend. The application utilizes the Gemini API for LLM capabilities and integrates observability via LangSmith.

2. Complete Folder Structure

```

/
... app/                                # Main FastAPI Backend Application
.  ... api/                             # API Routing and Endpoints
.  .  ... endpoints/                   # Route handlers (auth, chat, document, hitl)
.  .  ... deps.py                     # FastAPI Dependencies
.  ... graph/                           # LangGraph StateGraph Architecture
.  .  ... agent_nodes.py               # Specialist Agents (Orchestrator, Research, Coding, Tool)
.  .  ... checkpointer.py              # State persistence for HITL workflows
.  .  ... graph.py                     # Main workflow graph builder
.  .  ... hitl_nodes.py                # Human-in-the-Loop nodes
.  .  ... nodes.py                     # Core IO / RAG / Memory nodes
.  .  ... routing.py                   # Conditional edge routing logic
.  .  ... state.py                     # TypedDict defining the graph state
.  ... memory/                          # Long-term memory extraction and storage
.  .  ... service.py
.  ... models/                          # SQLAlchemy Database Models (User, Chat, Document, Memory...)
.  ... schemas/                        # Pydantic Schemas for Request/Response validation
.  ... services/                       # Core Business Logic Services
.  .  ... chat.py                      # Chat streaming and lifecycle
.  .  ... chunker.py                  # Document chunking algorithms
.  .  ... embeddings.py                # Vector embedding generation
.  .  ... gemini.py                    # LLM SDK Wrapper
.  .  ... pdf_parser.py                # PDF document ingestion
.  .  ... prompt_builder.py            # System prompts and templates
.  .  ... rag.py                       # RAG Search and augmentations
.  .  ... vectorstore.py               # ChromaDB integration
.  ... tools/                           # LLM Tool registry (calculate, wikipedia, time)
.  ... utils/                          # Utilities (Security, Hashing, JWT)
.  ... config.py                       # Environment configuration loading
.  ... database.py                     # SQLAlchemy Async Engine setup
.  ... main.py                         # FastAPI Application entry point
... frontend/                           # Next.js Frontend Application
.  ... src/
.  .  ... app/                         # Next.js App Router (page.jsx, layout.jsx, globals.css)
.  .  ... components/                  # React Components (Sidebar, ChatInput, HITLApprovalCard, ...)
.  .  ... lib/                         # Shared utilities (supabase.js)
.  .  ... styles/                       # Modular CSS styles
.  ... package.json
.  ... next.config.mjs
... chroma_db/                          # Local vector storage volume
... docker-compose.yml                  # Container orchestration
... Dockerfile                          # Backend Dockerfile
... requirements.txt                    # Python dependencies
... .env                                # Environment variables

```

3. Every File with its purpose

Backend (`app/`)

- `main.py`: Initializes the FastAPI app, configures CORS, mounts routers, connects to databases, and initializes the LangGraph on startup.
- `config.py`: Defines Settings using Pydantic, loading environments variables

(Database, JWT, Gemini, LangSmith).

- `database.py`: Configures the asyncpg SQLAlchemy engine and provides database session generators.
- `api/deps.py`: Contains dependency injections like `get_db` and `get_current_user`.
- `api/endpoints/auth.py`: Provides registration, OAuth2 login, and `/me` routes for authentication.
- `api/endpoints/chat.py`: Handles chat session CRUD and streaming LLM responses via Server-Sent Events (SSE).
- `api/endpoints/document.py`: Manages document uploads, PDF parsing, and integration into the RAG pipeline.
- `api/endpoints/hitl.py`: Endpoints for humans to approve, edit, or reject pending AI actions.
- `graph/state.py`: Defines the `GraphState` TypedDict holding the conversation history, active agent, memory, and HITL flags.
- `graph/graph.py`: Compiles the nodes and conditional edges into a `LangGraph StateGraph`.
- `graph/agent_nodes.py`: Implements the Orchestrator, Research Agent, Coding Agent, and Tool Agent logic.
- `graph/hitl_nodes.py`: Analyzes generated plans and interrupts the graph if human approval is required before execution.
- `graph/nodes.py`: Core graph nodes for handling user input, RAG execution, and final answer generation.
- `graph/routing.py`: Contains functions that determine the next node based on the current state.
- `graph/checkpointer.py`: Uses `AsyncPostgresSaver` to persist graph state across HTTP requests, allowing pausing and resuming.
- `memory/service.py`: Analyzes conversations to extract and persist factual user preferences to PostgreSQL.
- `models/*`: SQLAlchemy definitions for `User`, `Chat`, `Message`, `UploadedDocument`, and `Memory`.
- `schemas/*`: Pydantic types for enforcing API request/response structures.
- `services/rag.py`: Orchestrates text chunking, embedding generation, vector ingestion, and augmented prompt building.
- `services/chunker.py`: Implements sentence-aware document chunking.
- `services/embeddings.py`: Uses Gemini to generate text embeddings.
- `services/vectorstore.py`: Interfaces directly with ChromaDB for similarity searches.
- `services/gemini.py`: Wraps the google-genai SDK for model invocation.
- `tools/registry.py`: Registers tools (`calculate`, `search_wikipedia`, `get_current_time`) mapping names to Python functions.

Frontend (`frontend/src/`)

- `app/page.jsx`: The main login/landing page.
- `app/chat/page.jsx`: The primary conversational interface handling SSE streaming and state.
- `components/Sidebar.jsx`: Renders chat history, auto-generated titles, and navigation.
- `components/ChatInput.jsx`: Handles user input, file attachments, and Web Speech API microphone integration.

- components/ChatMessage.jsx: Renders individual messages, markdown, and citations.
- components/HITLApprovalCard.jsx: Displays a UI block for users to approve or reject paused AI actions.
- components/UploadModal.jsx: Interface for uploading PDFs to the RAG system.
- components/AuthProvider.jsx: React Context provider managing user authentication state.

4. AI Architecture Diagram

```
graph TD
    User([User]) -->|HTTP Request| API[FastAPI Endpoints]
    API -->|Session & State| DB[(PostgreSQL)]
    API -->|Invoke| LangGraph[LangGraph StateMachine]

    LangGraph <-->|Retrieval| RAG[RAG Service]
    RAG <-->|Embeddings| ChromaDB[(ChromaDB)]

    LangGraph <-->|LLM Queries| Gemini[Gemini Pro API]
    LangGraph <-->|Observability| LangSmith[LangSmith Tracing]

    LangGraph -->|Interrupt| HITL[Human-in-the-Loop]
    HITL -.->|Approval| User
    HITL -->|Execute| Tools[Tool Registry]
```

5. RAG Pipeline

- Ingestion: PDFs uploaded via `/api/v1/document/upload` are parsed in `pdf_parser.py`.
- Chunking: `chunker_service` applies sentence-aware splitting (max 800 chars, 150 overlap).
- Embedding: `embeddings_service` calls Gemini to generate vector representations.
- Storage: Vectors and structural metadata (page numbers, filename, `user_id`) are indexed into ChromaDB.
- Retrieval: On query, `RAGService` generates a query embedding, hits ChromaDB with a `user_id` filter, deduplicates results, and builds an augmented prompt with citations. Caching is employed for repeated queries.

6. LangGraph Workflow

```

stateDiagram-v2
    [*] --> receive_user_input
    receive_user_input --> retrieve_memory
    retrieve_memory --> orchestrator_node

    orchestrator_node --> execute_rag_search: RAG Needed
    orchestrator_node --> tool_agent_node: Tool Needed
    orchestrator_node --> research_agent_node: Deep Research
    orchestrator_node --> coding_agent_node: Coding Task
    orchestrator_node --> detect_action_required: Direct Answer

    execute_rag_search --> orchestrator_node
    tool_agent_node --> orchestrator_node
    research_agent_node --> orchestrator_node
    coding_agent_node --> orchestrator_node

    detect_action_required --> human_approval_node: High Risk Action
    detect_action_required --> generate_final_answer: Safe Action

    human_approval_node --> execute_approved_action: User Approved
    execute_approved_action --> extract_memory

    generate_final_answer --> extract_memory
    extract_memory --> [*]

```

7. Multi-Agent Architecture

The system employs a multi-agent orchestration pattern via LangGraph:

- **Orchestrator Agent:** Acts as the router. Evaluates the user query and decides whether to consult a specialist agent or answer directly.
- **Tool Agent:** Specialized in selecting and executing strict JSON-schema bound tools.
- **Research Agent:** Optimized for multi-step reasoning and deep data extraction.
- **Coding Agent:** Specialized in writing, reviewing, and analyzing code.

8. Memory System

Implemented in `app/memory/service.py` and `extract_memory` node.

- **Extraction:** At the end of the graph, the AI analyzes the user's latest query to extract factual preferences (e.g., "I am allergic to peanuts").
- **Storage:** Facts are persisted to PostgreSQL in the Memory table.
- **Retrieval:** At the start of the graph (`retrieve_memory` node), existing facts are retrieved and injected into the Orchestrator's system prompt, allowing the AI to retain long-term context across multiple sessions.

9. Tool Calling

Tools are managed in `app/tools/registry.py`. They are standard Python functions exposed to Gemini:

- `calculate`: Evaluates mathematical expressions safely.
- `search_wikipedia`: Fetches summaries from Wikipedia.
- `get_current_time`: Returns the exact server time.

When the LLM requests a tool, `execute_tool_node` or `tool_agent_node` intercepts the request, runs the registered function, and returns the string output to the LLM.

10. Human-in-the-Loop (HITL)

High-risk actions generated by the AI trigger the HITL pipeline.

- **Detection**: `detect_action_required` identifies if the proposed tool call or action requires human consent.
- **Interrupt**: LangGraph raises an `Interrupt()` which halts execution. State is saved securely via `AsyncPostgresSaver` (`checkpoint.py`).
- **UI Feedback**: The frontend receives a status: paused SSE event, rendering the `HITLApprovalCard`.
- **Resumption**: The user reviews, edits, or rejects the action. They submit via the `/api/v1/hitl/approve` endpoint, which resumes the graph execution dynamically.

11. LangSmith Integration

LangSmith is integrated natively to trace LLM calls, tools, and graph progression.

Configured via:

- `LANGCHAIN_TRACING_V2=true`
- `LANGCHAIN_API_KEY`
- `LANGCHAIN_PROJECT`

The `@traceable(run_type="...")` decorator is used extensively across RAG, tools, and memory services to provide deep execution telemetry.

12. Authentication Flow

```
sequenceDiagram
    participant User
    participant NextJS
    participant FastAPI
    participant Postgres

    User->>NextJS: Submits Login Form (Email/Pass)
    NextJS->>FastAPI: POST /api/v1/auth/login
    FastAPI->>Postgres: Verify Email & Hashed Password
    Postgres-->>FastAPI: User Data
    FastAPI-->>NextJS: Returns JWT Bearer Token
    NextJS->>User: Stores Token, Redirects to /chat

    User->>FastAPI: Any Request + Authorization: Bearer <Token>
    FastAPI->>FastAPI: Validates JWT via deps.py
```

13. Database Schema

Table Name	Description	Key Fields
users	Registered users	id`, `email`, `hashed_passwor
chats	Chat threads	id`, `user_id`, `title`, `cre
messages	Chat history	id`, `chat_id`, `role`, `cont
uploaded_documents	RAG Files	id`, `user_id`, `filename`, `
memories	Extracted Facts	id`, `user_id`, `memory_type`

14. API Endpoints

Method	Endpoint	Purpose
POST	/api/v1/auth/register	Create a new user account
POST	/api/v1/auth/login	Authenticate and retrieve JWT
GET	/api/v1/auth/me	Fetch active user profile
POST	/api/v1/chat/	Create a new chat session
GET	/api/v1/chat/	List all user chat sessions
GET	/api/v1/chat/{chat_id}	Fetch chat history
POST	/api/v1/chat/{chat_id}/stream	Stream AI response to user pr
POST	/api/v1/document/upload	Ingest PDF to ChromaDB RAG
POST	/api/v1/hitl/approve	Approve/Resume paused graph n

15. Frontend Architecture

Built on Next.js (App Router) and React.

State is managed via React hooks (useState, useEffect). UI is styled using Tailwind CSS, featuring custom CSS modules (e.g., aurora.css, loginCard.css) for advanced aesthetics and micro-animations. It uses Server-Sent Events (SSE) to consume the streaming /stream endpoint, parsing JSON chunks to build real-time text, handle citations, and trigger HITL UI states seamlessly.

16. Backend Architecture

Built on FastAPI. It is an asynchronous orchestrator utilizing asyncpg for non-blocking PostgreSQL queries. The architecture is modular: separating Routers (api/), Business Logic (services/), Data Models (models/), and AI Orchestration (graph/). Middleware handles request isolation, structured JSON logging, and security headers. It is designed to scale horizontally using Gunicorn with Uvicorn workers.

17. Technologies Used

- Backend Framework: FastAPI, Pydantic
- Database: PostgreSQL (SQLAlchemy Async), ChromaDB

- AI SDKs: Google GenAI (Gemini), LangGraph, LangSmith
- Frontend: Next.js 14, React, Tailwind CSS, Lucide Icons
- DevOps: Docker, Docker Compose, Gunicorn

18. Environment Variables

- APP_ENV: Environment flag (development or production)
- DATABASE_URL: AsyncPG connection string
- POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB: PostgreSQL Config
- GEMINI_API_KEY: Google Gemini Key
- SECRET_KEY: JWT Encryption Key
- CHROMADB_HOST, CHROMADB_PORT: ChromaDB network settings
- LANGCHAIN_TRACING_V2, LANGCHAIN_API_KEY: Observability
- HITL_ENABLED, MEMORY_ENABLED: Feature flags

19. Installation Guide

- Clone the repository.
- Copy .env.example to .env and fill in your GEMINI_API_KEY and SECRET_KEY.
- Ensure Docker and Docker Compose are installed.
- Run the build command:

```
docker-compose up --build -d
```

- Access the frontend at <http://localhost:3000> and API docs at <http://localhost:8000/docs>.

20. Deployment Guide

For production deployment:

- Set APP_ENV=production in the .env.
- Secure the POSTGRES_PASSWORD and SECRET_KEY.
- In docker-compose.yml, ensure the backend utilizes the provided gunicorn command for worker concurrency.
- Deploy the infrastructure using a VM (AWS EC2, DigitalOcean) or a container service (AWS ECS). Route traffic through a reverse proxy like Nginx or Traefik to handle SSL termination.

21. Future Improvements

- Web Search Agent Integration: Connect the Research Agent directly to Tavily or Google Search APIs.
- Multi-Modal RAG: Expand document ingestion to handle images and parse complex tables from PDFs.
- Token Optimization: Implement semantic caching layer in front of the LLM to reduce redundant requests and costs.

